```
    create_table :orders do |t|
      t.datetime :order_date
      t.string   :order_number
      t.integer  :customer_id
    end

    add_index :orders, :customer_id
  end
end
```

If you create an association some time after you build the underlying model, you need to remember to create an `add_column` migration to provide the necessary foreign key.

3.3.2 Creating Join Tables for `has_and_belongs_to_many` Associations

If you create a `has_and_belongs_to_many` association, you need to explicitly create the joining table. Unless the name of the join table is explicitly specified by using the `:join_table` option, Active Record creates the name by using the lexical order of the class names. So a join between customer and order models will give the default join table name of "customers_orders" because "c" outranks "o" in lexical ordering.

The precedence between model names is calculated using the `<` operator for `String`. This means that if the strings are of different lengths, and the strings are equal when compared up to the shortest length, then the longer string is considered of higher lexical precedence than the shorter one. For example, one would expect the tables "paper_boxes" and "papers" to generate a join table name of "papers_paper_boxes" because of the length of the name "paper_boxes", but it in fact generates a join table name of "paper_boxes_papers" (because the underscore *'_' is lexicographically _less* than 's' in common encodings).

Whatever the name, you must manually generate the join table with an appropriate migration. For example, consider these associations:

```
class Assembly < ActiveRecord::Base
  has_and_belongs_to_many :parts
end

class Part < ActiveRecord::Base
  has_and_belongs_to_many :assemblies
end
```

These need to be backed up by a migration to create the `assemblies_parts` table. This table should be created without a primary key:

```
class CreateAssembliesPartsJoinTable < ActiveRecord::Migration
  def change
    create_table :assemblies_parts, id: false do |t|
      t.integer :assembly_id
      t.integer :part_id
    end

    add_index :assemblies_parts, :assembly_id
    add_index :assemblies_parts, :part_id
  end
end
```

We pass `id: false` to `create_table` because that table does not represent a model. That's required for the association to work properly. If you observe any strange behavior in a `has_and_belongs_to_many` association like mangled models IDs, or exceptions about conflicting IDs, chances are you forgot that bit.